WaterManagement

Memoria de prácticas

Pedro Jesús Cros Pérez
DNI: 78162647E

Sistemas Distribuidos · Curso académico 2026/2027
Grupo de prácticas: (34022) SISTEMAS DISTRIBUIDOS Gr. 02 - 2
Universidad de Alicante

Índice

1 Introducción **3**
1.1 Tecnologías . . . 3
1.2 Código fuente . . . 3

2 Arquitectura del sistema **4**
2.1 Visión general . . . 4
2.2 Comunicaciones . . . 5
2.3 Escenario de despliegue . . . 6
2.4 Arquitectura interna de cada aplicación . . . 7

3 Componentes desarrollados **7**
3.1 `wm-common`: librería compartida . . . 7
3.2 `WM_Central` . . . 8
3.3 `WM_WS_M`: Monitor de la estación . . . 10
3.4 `WM_WS_E`: Engine de la estación . . . 10
3.5 `WM_FO`: aplicación del operario de campo . . . 12

4 Protocolos de comunicación **13**
4.1 Protocolo de sockets . . . 13
4.2 Mensajes entre `WM_WS_M` y `WM_Central` . . . 14
4.3 Mensajes entre `WM_WS_E` y `WM_WS_M` . . . 14
4.4 Kafka . . . 15

5 Datos y estados **16**
5.1 Base de datos . . . 16
5.2 Estados de una estación . . . 16

6 Funcionamiento **18**
6.1 Casos de uso implementados . . . 18
6.2 Flujo completo de un riego . . . 19
6.3 Averías . . . 19
6.4 Caídas de los módulos . . . 20

7 Guía de despliegue **20**
7.1 Requisitos . . . 21
7.2 Compilación . . . 21
7.3 Parte cloud: Railway . . . 21
7.4 Laboratorio . . . 24

8 Resultados **27**
8.1 Pruebas automáticas 27
8.2 Pruebas del sistema desplegado 27

9 Aspectos técnicos destacables **34**

10 Conclusiones **34**

1. Introducción

WaterManagement es un sistema distribuido que simula la gestión centralizada del riego de los parques y jardines de una ciudad. Cada zona verde tiene una estación de riego (*Watering Station*, WS) que abre y cierra el paso del agua, mide el caudal y detecta averías. Los operarios de mantenimiento piden y siguen los riegos desde su propia aplicación, y una central gobierna todo el sistema: decide el estado de cada estación, autoriza los riegos y guarda su historial.

El objetivo de esta primera entrega (*Release 1*) es construir ese sistema con aplicaciones independientes que se comunican de dos formas distintas: mediante **sockets TCP** con un protocolo propio y mediante **streaming de eventos con Kafka**. La parte cloud (la central, Kafka y la base de datos) se despliega en Railway, y las estaciones y los operarios, en los equipos del laboratorio.

Esta memoria describe la arquitectura y los componentes desarrollados (apartados 2 a 6), la guía de despliegue que se sigue en la corrección (apartado 7) y los resultados obtenidos (apartado 8).

1.1. Tecnologías

Tecnología	Uso en el proyecto
Java 21 y Spring Boot 4.1	Las cuatro aplicaciones
Apache Kafka 4.3 (modo KRaft)	Streaming de eventos entre las aplicaciones
SQLite	Base de datos de `WM_Central`
Maven 3.9	Compilación y pruebas del proyecto multimódulo
Docker y Docker Compose	Compilación, ejecución y despliegue de todos los módulos
Railway	Despliegue en la nube de `WM_Central`, Kafka y SQLite
GitHub	Repositorio del código fuente

1.2. Código fuente

El código fuente está en el repositorio de GitHub https://github.com/pedrocrosua/water_management_network (rama `main`) y se entrega también junto a esta memoria. Es un proyecto Maven con cinco módulos:

Carpeta	Contenido
`wm-common/`	Librería compartida: protocolo de sockets, mensajes de Kafka y estados
`wm-central/`	Aplicación `WM_Central`
`wm-ws-monitor/`	Aplicación `WM_WS_M` (Monitor de la estación)
`wm-ws-engine/`	Aplicación `WM_WS_E` (Engine de la estación)
`wm-field-operator/`	Aplicación `WM_FO` (operario de campo)
`scripts/`	Scripts para lanzar los módulos en el laboratorio
`requests/`	Fichero de ejemplo con peticiones de riego
`Dockerfile`	Imagen común para las cuatro aplicaciones
`docker-compose*.yml`	Despliegue con Docker Compose
`lab.env`	Direcciones de `WM_Central` y Kafka en Railway

2. Arquitectura del sistema

2.1. Visión general

El sistema está formado por cuatro aplicaciones independientes y una librería común:

Módulo	Ejecutable	Tipo	Función
`wm-central`	`WM_Central`	Spring Boot web	Gobierna el sistema: estados, autorización de riegos, base de datos y panel de monitorización
`wm-ws-monitor`	`WM_WS_M`	Spring Boot de consola	Monitor de la estación: vigila al Engine y mantiene la estación registrada en la central
`wm-ws-engine`	`WM_WS_E`	Spring Boot de consola (con menú)	Engine de la estación: electroválvula, caudalímetro y sensor de averías simulados

Módulo	Ejecutable	Tipo	Función
`wm-field-operator`	`WM_FO`	Spring Boot de consola (con menú)	Aplicación del operario de campo: consulta las estaciones y pide riegos
`wm-common`	—	Librería	Código compartido por las aplicaciones

Una estación de riego es la pareja formada por un `WM_WS_M` y un `WM_WS_E`. Los nombres de los ejecutables son los que exige el enunciado: cada módulo genera un JAR con ese nombre (`WM_Central.jar`, `WM_WS_M.jar`, `WM_WS_E.jar` y `WM_FO.jar`).

2.2. Comunicaciones

Se usan dos mecanismos de comunicación, cada uno para lo que mejor resuelve:

- **Sockets TCP con protocolo propio** para el control de las estaciones: el registro y la autenticación del Monitor en la central, los latidos de salud y los avisos de avería. Son conversaciones punto a punto en las que hay que saber al momento si el otro extremo se ha caído, y cada mensaje se confirma de forma explícita.
- **Kafka** para todo lo relacionado con los riegos: peticiones, órdenes a las estaciones, telemetría, notificaciones a los operarios y estado de las estaciones. Son eventos asíncronos que pueden leer varias aplicaciones a la vez, y se pueden añadir productores y consumidores sin modificar los que ya existen.

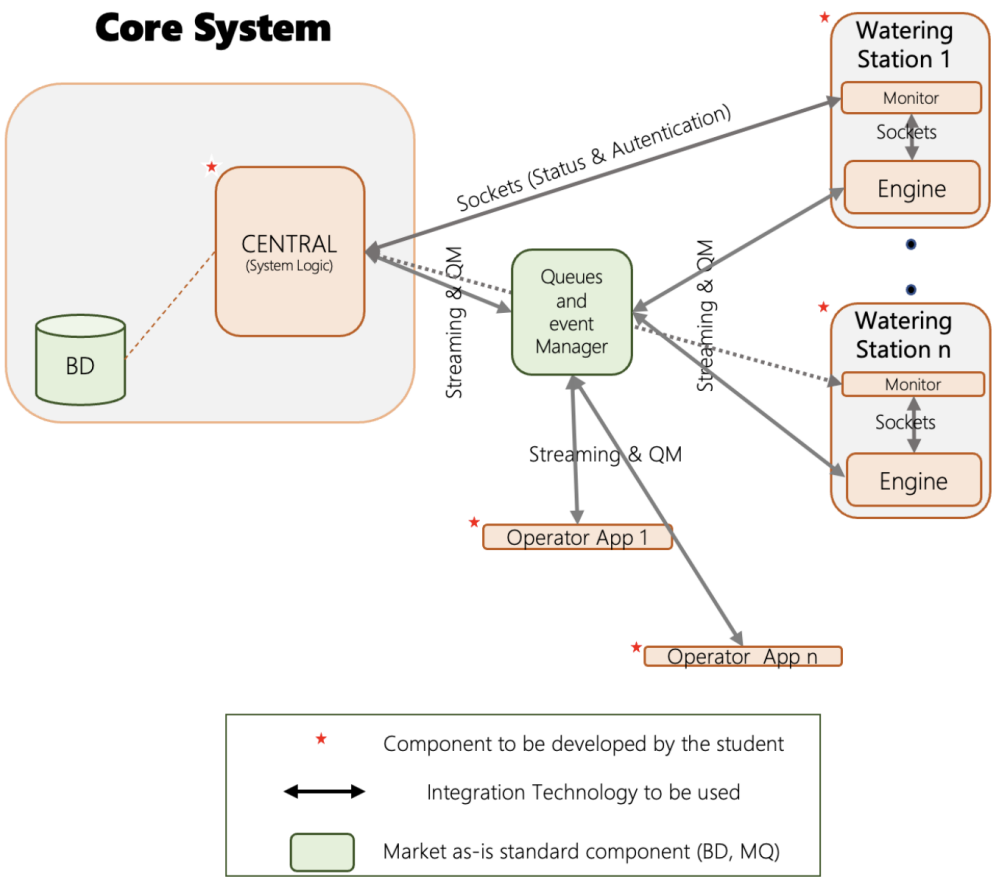


Figura 1: Esquema de comunicaciones del sistema: sockets para el control de las estaciones y Kafka para los riegos.

`WM_FO` y `WM_WS_E` nunca abren un socket contra `WM_Central`. El único socket que llega a la central es el del Monitor, que actúa como pasarela de control de su estación.

El esquema es genérico. En esta implementación el Monitor no usa Kafka, así que la línea punteada que le llega desde Kafka no se usa: el Monitor solo se comunica por sockets, con la central y con su Engine.

2.3. Escenario de despliegue

```
 RAILWAY (nube)                      LABORATORIO
+-----------------------------+      PC de las estaciones
|  WM_Central + SQLite (vol.) |        WS-01 = WM_WS_M + WM_WS_E
|  Kafka (KRaft)              |        WS-02 = WM_WS_M + WM_WS_E
+-----------------------------+        WS-03 = WM_WS_M + WM_WS_E
```

```
                                    PC de los operarios
                                      WM_FO (FO-001)
                                      WM_FO (FO-002)
```

Todos los módulos se ejecutan en contenedores Docker. Los PCs del laboratorio solo necesitan Docker y git, y se conectan a Railway a través de Internet. Se pueden lanzar tantas estaciones y operarios como se quiera, en el mismo PC o en varios.

2.4. Arquitectura interna de cada aplicación

Las cuatro aplicaciones siguen una arquitectura hexagonal sencilla, con tres capas que separan la lógica de la tecnología que la rodea:

Capa	Contenido	Regla
`domain`	Objetos del problema: `Station`, `Valve`, `Irrigation`...	No conoce Kafka, sockets ni SQL
`application`	Lógica de los casos de uso y sus puertos (interfaces)	Solo usa interfaces
`infrastructure`	Adaptadores: Kafka, sockets, SQLite, web, consola y configuración	Implementan o usan los puertos

Las dependencias siempre van hacia dentro: `infrastructure` depende de `application`, y `application` de `domain`. Por ejemplo, `IrrigationService` (aplicación) envía una orden a una estación a través del puerto `StationCommandSender`, y quien la publica en Kafka es el adaptador `KafkaStationCommandSender`. Gracias a esta separación, la lógica se prueba con implementaciones falsas en memoria, sin red ni base de datos.

3. Componentes desarrollados

3.1. `wm-common`: librería compartida

Reúne el código que usan varias aplicaciones, para que todas hablen exactamente el mismo idioma:

Paquete	Clases	Responsabilidad
`protocol`	`FrameCodec`, `FrameReader`, `FrameChannel`, `SocketMessage`, `MessageType`	Protocolo de sockets: construir y comprobar tramas, leerlas del flujo TCP y confirmarlas con ACK/NACK
`event`	`Topics`, `IrrigationRequest`, `StationCommand`, `StationEvent`, `CommandAck`, `Telemetry`, `IrrigationEnded`, `OperatorNotification`, `StationStatus`	Mensajes de Kafka, que viajan como JSON
`model`	`StationState`, `EndReason`, `IrrigationSource`	Estados de una estación, motivos de fin de un riego y origen de una petición
`validation`	`Validator`	Comprobaciones de entrada reutilizables

3.2. `WM_Central`

Es el cerebro del sistema y el único que decide el estado de las estaciones. Sus funciones son:

- Ofrecer el servidor de sockets al que se conectan los Monitores: da de alta las estaciones nuevas, las autentica y vigila sus latidos de salud.
- Recibir por Kafka las peticiones de riego, validarlas, ordenar a la estación que riegue y avisar al operario de cada paso.
- Recibir la telemetría de los riegos y guardar en SQLite cada riego terminado.
- Mostrar un panel web de monitorización que se actualiza solo, mediante eventos enviados por el servidor (SSE). Desde el panel se dan órdenes a una estación o a todas: regar, bloquear y activar.

Arranque (el panel web atiende en el puerto 8080):

```
WM_Central --socket.port=5000 --db.path=/data/wm.db \
           --spring.kafka.bootstrap-servers=<host>:<puerto>
```

Capa	Clases	Responsabilidad
Dominio	`Station`, `Operator`, `IrrigationRecord`, `OngoingIrrigation`	Estación, operario, riego terminado y riego en marcha
Aplicación	`StationRegistry`	Estado de cada estación: cada cambio se guarda y se comunica al panel, al Monitor y a Kafka
	`IrrigationService`	Valida y autoriza los riegos, sigue su progreso, los corta ante una avería o un bloqueo y los guarda al terminar
	`CentralOrders`	Órdenes del panel: regar, bloquear y activar una estación o todas
Sockets	`StationSocketServer`, `MonitorSession`, `MonitorSessions`	Servidor de sockets y conversación con cada Monitor
Kafka	`IrrigationRequestListener`, `StationEventListener`, `KafkaStationCommandSender`, `KafkaOperatorNotifier`, `KafkaStationStatusPublisher`, `KafkaTopics`	Lectura de peticiones y de eventos de las estaciones; envío de órdenes, notificaciones y estados; creación de los topics
Persistencia	`JdbcStationRepository`, `JdbcOperatorRepository`, `JdbcIrrigationRepository`, `JdbcOngoingIrrigationRepository`	Acceso a SQLite con SQL explícito
Web	`StationController`, `OrderController`, `PanelBroadcaster`	Datos del panel, órdenes (`/api/orders/...`) y envío de los cambios al navegador

El panel está hecho con HTML, CSS y JavaScript sin librerías (`index.html`, `panel.css` y `panel.js`).

3.3. `WM_WS_M`: Monitor de la estación

Mantiene la estación registrada y conectada a la central, y vigila la salud de su Engine:

- Al arrancar, se autentica en la central con el identificador y la ubicación de la estación (`AUTH`). Si el identificador es nuevo, la central da de alta la estación.
- Cada segundo envía un latido de salud a la central (`HEALTH`) y un `PING` a su Engine.
- Si el Engine responde `KO` o deja de responder, avisa a la central de una avería (`LEAK`). Cuando el Engine vuelve a responder bien, envía `LEAK_CLEARED`.
- Muestra el estado de la estación que le comunica la central, por ejemplo «VERDE - Disponible» o «NARANJA - Fuera de Servicio».
- Si pierde la conexión con la central, muestra «Imposible conectar con WM_Central» y vuelve a intentarlo solo.

Arranque:

```
WM_WS_M --engine.port=6000 --central.host=<host> --central.port=<puerto> \
        --station.id=WS-01 --station.location="Parque de Canalejas"
```

Capa	Clases	Responsabilidad
Dominio	`StationIdentity`	Identificador y ubicación de la estación
Aplicación	`StationMonitor`	Autenticación y latidos hacia la central; decide cuándo avisar de una avería
Infraestructura	`SocketCentralConnection`	Conexión por socket con la central
	`EngineSocketServer`, `EngineSession`	Servidor de sockets para el Engine: `HELLO/ID` y `PING/PONG`
	`MonitorConfiguration`	Lee los argumentos y arranca el Monitor

3.4. `WM_WS_E`: Engine de la estación

Simula la parte física de la estación: la electroválvula, el caudalímetro y el sensor de averías.

- Se conecta a su Monitor (`HELLO`) y recibe de él el identificador de la estación (`ID`). Como pide el enunciado, el Engine no recibe el identificador por parámetro.
- Recibe por Kafka las órdenes de la central (START y STOP), abre o cierra la válvula y confirma cada orden.
- Mientras riega, publica cada segundo el caudal (unos 20 litros por minuto, simulados) y los litros acumulados. Al llegar al límite de tiempo, cierra la válvula y publica el fin del riego.
- Guarda cada segundo lo regado en el fichero `data/irrigation.json`, que está en un volumen de Docker: si el Engine se cae, al volver envía a la central el estado final del riego.
- Tiene un menú de consola para el técnico de la estación:

Opción	Acción
1	Simular una avería (el Engine pasa a responder KO)
2	Reparar la avería
3	Parar el riego en curso
4	Pedir un riego manual, que tiene que autorizar la central

Arranque:

```
WM_WS_E --monitor.host=<host> --monitor.port=6000 \
        --spring.kafka.bootstrap-servers=<host>:<puerto>
```

Capa	Clases	Responsabilidad
Dominio	`Valve`, `LeakSensor`, `Irrigation`, `StationIdentity`	Válvula, sensor de averías, riego en curso e identidad recibida del Monitor
Aplicación	`IrrigationController`	Ejecuta las órdenes, lleva el reloj del riego y publica la telemetría
	`MonitorLink`	Mantiene la conexión con el Monitor; si se pierde, la estación queda «Averiada»
	`ManualIrrigation`	Riego manual pedido desde el menú

Capa	Clases	Responsabilidad
Infraestructura	`SocketMonitorConnection`	Conexión por socket con el Monitor
	`StationCommandListener`, `KafkaStationEventPublisher`, `KafkaIrrigationRequestSender`	Lectura de órdenes y publicación de eventos y de peticiones de riego manual
	`FileIrrigationBackup`, `SimulatedFlowMeter`, `EngineConsole`	Copia del riego en curso, caudalímetro simulado y menú

3.5. `WM_FO`: aplicación del operario de campo

Es una aplicación de consola con este menú:

Opción	Acción
1	Listar las estaciones con su estado
2	Pedir un riego: estación y duración en segundos
3	Pedir los riegos de un fichero, por ejemplo `requests/example.csv`

- Publica las peticiones en Kafka y muestra cada paso que le notifica la central (`RECEIVED`, `VALIDATING`, `AUTHORIZATION_REQUESTED` y `AUTHORIZED`, o `DENIED` con el motivo), el progreso del riego segundo a segundo y el resumen final: litros, duración y motivo de fin.
- En el menú, si la central no contesta a una petición en 10 segundos, muestra «Imposible conectar con WM_Central».
- En el modo fichero, cada línea es una petición con el formato `estación,segundos` (se saltan las líneas vacías y las que empiezan por #). Envía una petición y espera a que termine o se deniegue, como mucho la duración pedida más 30 segundos. Después espera 4 segundos y pasa a la siguiente.

Arranque:

```
WM_FO --operator.id=FO-001 --spring.kafka.bootstrap-servers=<host>:<puerto>
```

Capa	Clases	Responsabilidad
Dominio	`StationBoard`	Último estado conocido de cada estación
Aplicación	`IrrigationRequests`	Envía las peticiones y espera la respuesta de la central
	`RequestFile`	Modo fichero
Infraestructura	`OperatorConsole`	Menú de consola
	`KafkaIrrigationRequestSender`	Publicación de las peticiones
	`OperatorNotificationListener`, `IrrigationTelemetryListener`, `StationStatusListener`	Notificaciones del operario, progreso de sus riegos y estado de las estaciones

4. Protocolos de comunicación

4.1. Protocolo de sockets

Todas las conexiones por socket usan el mismo protocolo, implementado en `wm-common`:

```
<STX><DATA><ETX><LRC>
```

- STX (`0x02`) y ETX (`0x03`) delimitan el mensaje.
- DATA es texto ASCII imprimible con los campos separados por `#`, por ejemplo `AUTH#WS-01#Parque de Canalejas`.
- LRC es un byte de control: el XOR de todos los bytes desde el primero de DATA hasta ETX, ambos incluidos.
- El receptor comprueba la trama y responde con un único byte: ACK (`0x06`) si es correcta o NACK (`0x15`) si no lo es.
- Si el emisor recibe NACK o no recibe respuesta en 2 segundos, reenvía la trama hasta 3 veces. Si se agotan los reintentos, da la conexión por caída.

`FrameCodec` construye y comprueba las tramas. `FrameReader` separa las tramas completas del flujo de bytes de TCP, porque una trama puede llegar en varios trozos o varias tramas juntas. `FrameChannel` gestiona los envíos con su confirmación y sus reintentos.

4.2. Mensajes entre `WM_WS_M` y `WM_Central`

El Monitor es el cliente y la central, el servidor.

Sentido	Mensaje	Cuándo
Monitor → Central	`AUTH#<id>#<ubicación>`	Al arrancar el Monitor
Central → Monitor	`AUTH_OK#<id>#<estado>`	Registro aceptado; si la estación es nueva, queda dada de alta
Central → Monitor	`AUTH_KO#<motivo>`	Mensaje de registro no válido
Monitor → Central	`HEALTH#<id>#OK`	Cada segundo
Monitor → Central	`LEAK#<id>`	El Engine responde KO o ha dejado de responder
Monitor → Central	`LEAK_CLEARED#<id>`	La avería se ha resuelto
Central → Monitor	`STATE#<id>#<estado>`	La central comunica un cambio de estado de la estación

Si la central no recibe nada del Monitor durante 3 segundos, o se cierra el socket, la estación pasa a `DISCONNECTED`.

4.3. Mensajes entre `WM_WS_E` y `WM_WS_M`

El Engine es el cliente y el Monitor, el servidor.

Sentido	Mensaje	Cuándo
Engine → Monitor	`HELLO`	Al conectar
Monitor → Engine	`ID#<id>#<ubicación>`	El Engine adopta el identificador de su Monitor
Monitor → Engine	`PING#<n>`	Cada segundo
Engine → Monitor	`PONG#<n>#OK`	Funcionamiento normal
Engine → Monitor	`PONG#<n>#KO`	Avería simulada desde el menú del Engine

Un solo PONG con KO basta para que el Monitor avise de la avería. Tres `PING` seguidos sin respuesta, o la caída de la conexión, se tratan igual: la estación se da por averiada.

4.4. Kafka

Hay cinco topics, que crea la central al arrancar, con tres particiones cada uno. Los mensajes viajan como JSON y se definen en `wm-common`.

Topic	Lo publica	Lo leen	Contenido (clave)
`wm.irrigation.requests`	`WM_FO` y `WM_WS_E` (riego manual)	`WM_Central`	Peticiones de riego (estación)
`wm.station.commands`	`WM_Central`	Cada `WM_WS_E`	Órdenes `START`, `STOP` y `DENIED` (estación)
`wm.station.telemetry`	`WM_WS_E`	`WM_Central` y `WM_FO`	Confirmación de las órdenes, caudal y litros cada segundo, y fin del riego (estación)
`wm.operator.notifications`	`WM_Central`	Cada `WM_FO`	Pasos de la petición, autorización o denegación, y resumen final (operario)
`wm.station.status`	`WM_Central`	Cada `WM_FO`	Estado de todas las estaciones (estación)

Grupos de consumidores. La central lee con un único grupo (`central`): si se lanzaran varias centrales, Kafka repartiría entre ellas las particiones y, con ellas, el trabajo (patrón *Competing Consumers*). En cambio, cada Engine y cada operario usan un grupo propio (`engine-WS-01`, `fo-FO-001`…). Así, cada uno recibe todos los mensajes del topic y se queda solo con los suyos: el Engine, con las órdenes cuya clave es su estación, y el operario, con las notificaciones cuya clave es su identificador y con la telemetría cuyo campo `operatorId` es el suyo. Si compartieran grupo, una orden para `WS-01` podría llegar al Engine de `WS-02`.

Kafka guarda para cada grupo por dónde iba leyendo, así que un consumidor que se reinicia continúa donde lo dejó. Solo la primera vez que arranca un grupo empieza por los mensajes nuevos (`auto-offset-reset=latest`). Hay dos excepciones: la central salta siempre al final de `wm.irrigation.requests` al arrancar, para no atender peticiones antiguas, y la lista de estaciones del operario se reconstruye en cada arranque leyendo `wm.station.status` desde el principio.

5. Datos y estados

5.1. Base de datos

`WM_Central` guarda sus datos en un fichero SQLite (`wm.db`) que vive en un volumen, para que sobreviva a los reinicios y a los nuevos despliegues. Al arrancar, crea las tablas si no existen (`schema.sql`) y carga los datos iniciales (`data.sql`): los operarios `FO-001` y `FO-002` y las estaciones `WS-01`, `WS-02` y `WS-03`.

Tabla	Contenido
`stations`	Estaciones: identificador, ubicación, estado y si está bloqueada
`operators`	Operarios dados de alta y si están activos
`irrigations`	Historial de riegos terminados: estación, operario, origen, inicio, fin, límite de tiempo, litros y motivo de fin
`ongoing_irrigations`	Riegos en marcha, para no perderlos si la central se cae
`audit_events`	Preparada para un registro de eventos; no se usa en esta versión

La telemetría de cada segundo no se guarda: se muestra en el panel y al operario, y al terminar el riego se escribe una sola fila con el resumen. Al arrancar, la central pone todas las estaciones en `DISCONNECTED` hasta que se conecta su Monitor, porque hasta entonces no puede conocer su estado real. El bloqueo de una estación, en cambio, se conserva entre reinicios.

5.2. Estados de una estación

Estado	Color en el panel	Significado
`DISCONNECTED`	Gris	La central no recibe mensajes de su Monitor
`AVAILABLE`	Verde	Operativa y en reposo
`IRRIGATING`	Verde parpadeante	Regando: el panel muestra el caudal, los litros y el operario
`LEAK`	Rojo	Averiada
`OUT_OF_SERVICE`	Naranja	Bloqueada desde el panel de la central

Desde	Hasta	Cuándo
DISCONNECTED	AVAILABLE	El Monitor se autentica y la estación no está bloqueada
DISCONNECTED	OUT_OF_SERVICE	El Monitor se autentica y la estación está bloqueada
AVAILABLE	IRRIGATING	La central autoriza un riego y el Engine confirma que abrió la válvula
IRRIGATING	AVAILABLE	El riego termina por tiempo o porque se para desde la estación
AVAILABLE o IRRIGATING	LEAK	El Monitor avisa de una avería; si había un riego, se corta
LEAK	AVAILABLE	El Monitor avisa de que la avería se ha resuelto y la estación no está bloqueada
LEAK	OUT_OF_SERVICE	El Monitor avisa de que la avería se ha resuelto y la estación está bloqueada
AVAILABLE o IRRIGATING	OUT_OF_SERVICE	Bloqueo desde el panel; si había un riego, se corta
OUT_OF_SERVICE	AVAILABLE	Activación desde el panel
Cualquiera	DISCONNECTED	Se cierra el socket del Monitor o pasan 3 segundos sin noticias suyas

Solo la central calcula el estado: el Monitor y el Engine informan de hechos (salud, averías y caudal), nunca de estados. Solo se autoriza un riego en una estación AVAILABLE. Con esto se cumple la tabla de estados de la guía de corrección:

Monitor	Engine	Estado en la central
Funciona	Funciona	AVAILABLE: disponible (verde)
Funciona	Caído o averiado	LEAK: averiada (rojo)
Caído	Funciona	DISCONNECTED: desconectada
Caído	Caído	DISCONNECTED: desconectada

6. Funcionamiento

6.1. Casos de uso implementados

Caso de uso	Lo inicia	Resumen
CU-01. Conectar una estación	El sistema	El Engine se identifica ante su Monitor y el Monitor se autentica en la central; la estación pasa a `AVAILABLE`
CU-02. Vigilar la salud	El reloj	`PING`/`PONG` cada segundo entre Monitor y Engine, y `HEALTH` hacia la central
CU-03. Simular y reparar una avería	El técnico de la estación	Opciones 1 y 2 del Engine: avería (`LEAK`) y reparación
CU-04. Consultar las estaciones	El operario	Opción 1 de `WM_FO`
CU-05. Solicitar un riego	El operario	Opción 2 de `WM_FO`: la central valida, ordena el riego y notifica cada paso
CU-06. Seguir el progreso	El operario y la central	Caudal y litros cada segundo, en el panel y en el `WM_FO` que lo pidió
CU-07. Terminar un riego	El reloj o el técnico	Fin por tiempo o con la opción 3 del Engine; se guarda y se envía el resumen
CU-08. Monitorizar la red	El operador de la central	Panel web con el estado de todas las estaciones, actualizado al momento
CU-09. Cortar un riego por avería	El sistema	Una avería durante un riego lo corta, lo guarda y avisa al operario
CU-10. Bloquear y activar estaciones	El operador de la central	Botones del panel; una estación bloqueada no admite riegos
CU-11. Ordenar un riego desde la central	El operador de la central	Botones `Regar` y `Regar todas` del panel
CU-12. Peticiones desde un fichero	El operario	Opción 3 de `WM_FO`
Riego manual	El técnico de la estación	Opción 4 del Engine; la central tiene que autorizarlo

6.2. Flujo completo de un riego

1. El operario elige la opción 2 en `WM_FO` e indica la estación y la duración. La petición se publica en `wm.irrigation.requests`.
2. La central la recibe y comprueba que la estación existe y está disponible, que el operario existe y está activo, y que la estación no está atendiendo otra petición. Avisa al operario de cada paso: `RECEIVED` y `VALIDATING`.
3. Si alguna comprobación falla, el operario recibe `DENIED` con el motivo.
4. Si todo es correcto, la central publica la orden `START` en `wm.station.commands` y avisa al operario con `AUTHORIZATION_REQUESTED`.
5. El Engine de esa estación abre la válvula y confirma la orden. La central pasa la estación a `IRRIGATING` y avisa al operario con `AUTHORIZED`. Si el Engine no confirma en 10 segundos, la petición se deniega.
6. Cada segundo, el Engine publica el caudal y los litros acumulados, que se ven en el panel de la central y en la consola del operario.
7. Al cumplirse el tiempo, o si el técnico para el riego con la opción 3, el Engine cierra la válvula y publica el fin del riego.
8. La central guarda el riego en SQLite, devuelve la estación a `AVAILABLE` y envía al operario el resumen: litros, duración y motivo de fin.

Las peticiones que llegan desde el panel de la central o desde el menú de la estación (riego manual) siguen el mismo camino.

6.3. Averías

1. El técnico elige la opción 1 en el Engine, que empieza a responder `PONG#...#KO`.
2. El Monitor avisa a la central (`LEAK`) y la estación pasa a rojo en el panel.
3. Si estaba regando, la central ordena cerrar la válvula (`STOP` con motivo `LEAK`), guarda el riego con los litros del último segundo recibido y avisa al operario. No espera la respuesta del Engine, que podría estar caído.
4. Con la opción 2 se repara la avería: el Monitor envía `LEAK_CLEARED` y la estación vuelve a estar disponible (`AVAILABLE`), o fuera de servicio (`OUT_OF_SERVICE`) si está bloqueada.

6.4. Caídas de los módulos

Se cae	Qué ocurre	Al volver a lanzarlo
`WM_WS_E`	El Monitor deja de recibir respuesta y avisa de una avería: la estación pasa a rojo y, si regaba, la central corta el riego, lo guarda y avisa al operario. El Engine había guardado cada segundo lo regado en un volumen de Docker	El Engine envía a la central el estado final del riego interrumpido. El Monitor vuelve a recibir respuesta y la estación vuelve a estar disponible
`WM_WS_M`	La central marca la estación como desconectada (gris) y corta el riego si lo había. El Engine cierra la válvula, muestra «Averiada» y no admite riegos	El Monitor se autentica de nuevo y el Engine se reconecta solo: la estación vuelve a estar disponible
`WM_FO`	Los riegos siguen: ni la central ni las estaciones dependen del operario	Kafka le entrega las notificaciones publicadas mientras estaba caído, así que el operario ve el resultado de su riego
`WM_Central`	Las estaciones terminan los riegos en curso, porque el Engine no necesita a la central para regar. No se admiten peticiones nuevas: los Monitores muestran «Imposible conectar con WM_Central», y un operario que pide un riego recibe el mismo aviso a los 10 segundos	La central recupera de SQLite los riegos que estaban en marcha y procesa su final, que la espera en Kafka. Los Monitores se reconectan solos

En todos los casos solo hay que volver a lanzar el módulo caído: el resto del sistema se recupera solo y no se pierde ningún riego.

7. Guía de despliegue

Esta es la guía que se sigue el día de la corrección. La parte cloud (`WM_Central`, Kafka y SQLite) llega ya desplegada en Railway, y en el laboratorio solo hay que clonar el repositorio y lanzar los módulos. No se usa ningún entorno de desarrollo: todo se compila y se ejecuta con Docker.

7.1. Requisitos

- En los PCs del laboratorio: Docker (con Docker Compose) y git. No hace falta instalar Java ni Maven.
- Para la parte cloud: una cuenta de Railway con acceso al repositorio de GitHub.

7.2. Compilación

Las cuatro aplicaciones se construyen con un único `Dockerfile` de dos etapas:

1. **Compilación.** Una imagen con Maven y Java 21 compila el módulo elegido con el argumento `MODULE`, junto con `wm-common`, y genera su JAR (`WM_Central.jar`, `WM_WS_M.jar`, `WM_WS_E.jar` o `WM_FO.jar`).
2. **Ejecución.** Una imagen ligera, con solo el entorno de ejecución de Java 21, copia ese JAR y lo arranca.

Railway y los scripts del laboratorio construyen las imágenes automáticamente. Para compilar todo el proyecto y pasar las pruebas sin instalar nada en el equipo:

```
docker compose run --rm maven mvn -B verify
```

Los JAR quedan en la carpeta `target/` de cada módulo, por ejemplo `wm-central/target/WM_Central.jar`.

7.3. Parte cloud: Railway

Se crea un proyecto en Railway con dos servicios en el mismo entorno, para que se vean por la red privada de Railway:

Servicio	`kafka`	`wm-central`
Origen	Imagen `apache/kafka:4.3.1`	Repositorio de GitHub (`Dockerfile`)
Puertos internos	9092 (red privada) y 29092 (externo)	8080 (panel) y 5000 (sockets)
Acceso público	TCP Proxy al 29092	Dominio HTTP al 8080 y TCP Proxy al 5000
Volumen	`/var/lib/kafka/data`	`/data`

Servicio `kafka`

1. *Create* → *Docker Image* con la imagen `apache/kafka:4.3.1`. El servicio se llama `kafka`.
2. En *Settings* → *Networking* se añade un *TCP Proxy* al puerto `29092`, antes del primer despliegue. Railway asigna una dirección pública del tipo `xxx.proxy.rlwy.net:12345`, que es la que usan los PCs del laboratorio.
3. Se añade un volumen montado en `/var/lib/kafka/data`, para que los topics y los mensajes sobrevivan a los reinicios.
4. Se definen las variables del servicio:

```
RAILWAY_RUN_UID=0
KAFKA_NODE_ID=1
KAFKA_PROCESS_ROLES=broker,controller
KAFKA_CONTROLLER_QUORUM_VOTERS=1@localhost:9093
KAFKA_LISTENERS=INTERNAL://:9092,CONTROLLER://:9093,EXTERNAL://:29092
KAFKA_ADVERTISED_LISTENERS=INTERNAL://${{RAILWAY_PRIVATE_DOMAIN}}:9092,\
EXTERNAL://${{RAILWAY_TCP_PROXY_DOMAIN}}:${{RAILWAY_TCP_PROXY_PORT}}
KAFKA_LISTENER_SECURITY_PROTOCOL_MAP=INTERNAL:PLAINTEXT,CONTROLLER:PLAINTEXT,\
EXTERNAL:PLAINTEXT
KAFKA_INTER_BROKER_LISTENER_NAME=INTERNAL
KAFKA_CONTROLLER_LISTENER_NAMES=CONTROLLER
KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR=1
KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR=1
KAFKA_TRANSACTION_STATE_LOG_MIN_ISR=1
KAFKA_NUM_PARTITIONS=3
KAFKA_LOG_DIRS=/var/lib/kafka/data/kafka-logs
KAFKA_HEAP_OPTS=-Xms512m -Xmx512m
```

(Las dos variables partidas con `\` se escriben en una sola línea.)

Kafka se ejecuta en modo KRaft, sin ZooKeeper, con tres *listeners*: `CONTROLLER`, que usa el propio Kafka para el modo KRaft, y dos para los clientes. Por `INTERNAL` se conecta la central dentro de la red privada de Railway; `EXTERNAL` se anuncia con la dirección del TCP Proxy y es el que usan los módulos del laboratorio. Es el punto clave del despliegue: un cliente de Kafka se conecta primero a la dirección que se le indica, pero después usa la que le anuncia el broker; si esa dirección no fuera alcanzable desde el laboratorio, el cliente se quedaría colgado sin un error claro.

Además, `RAILWAY_RUN_UID=0` permite a Kafka escribir en el volumen, `KAFKA_LOG_DIRS` guarda los datos en una subcarpeta del volumen (Kafka no arranca si encuentra en su carpeta otros ficheros, como el `lost+found` del volumen) y `KAFKA_HEAP_OPTS` limita su memoria a 512 MB.

Servicio `wm-central`

1. *Create* → *GitHub Repo* con el repositorio del proyecto. El servicio se llama `wm-central`, y Railway construye su imagen con el `Dockerfile` de la raíz.
2. Se definen las variables del servicio:

```
MODULE=wm-central
APP_NAME=WM_Central
SERVER_PORT=8080
SOCKET_PORT=5000
DB_PATH=/data/wm.db
SPRING_KAFKA_BOOTSTRAP_SERVERS=${{kafka.RAILWAY_PRIVATE_DOMAIN}}:9092
```

`MODULE` y `APP_NAME` eligen qué aplicación construye el `Dockerfile`. Las demás son la configuración de la central: Spring Boot las lee como las propiedades `server.port`, `socket.port`, `db.path` y `spring.kafka.bootstrap-servers`.

3. Se añade un volumen montado en `/data`, donde se guarda la base de datos `wm.db`.
4. En *Settings* → *Networking* se genera un dominio público para el puerto `8080` (el panel) y se añade un *TCP Proxy* para el puerto `5000` (el servidor de sockets de los Monitores).

Una vez desplegado, el sistema queda accesible en estas direcciones:

Recurso	Dirección
Panel de `WM_Central`	https://wm-central-production-fb0a.up.railway.app
Sockets de `WM_Central` (para los `WM_WS_M`)	`iriguchi.proxy.rlwy.net:26572`
Kafka (para los `WM_WS_E` y los `WM_FO`)	`altaria.proxy.rlwy.net:55723`

Estas direcciones están en el fichero `lab.env` del repositorio, que leen los scripts del laboratorio. Si cambiaran en Railway, basta con actualizarlas en ese fichero.

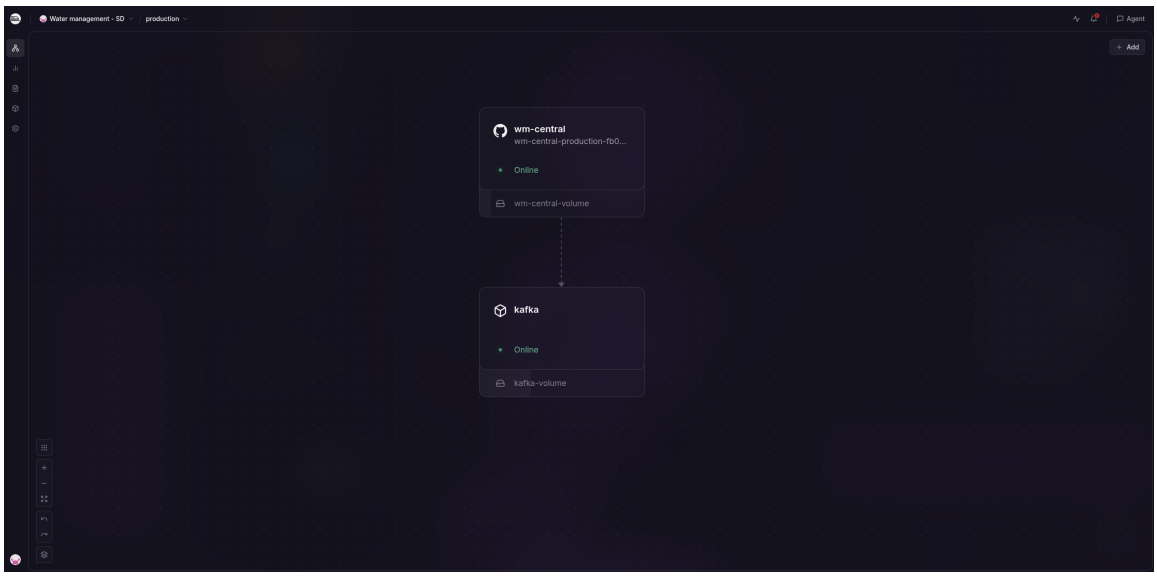


Figura 2: Servicios `kafka` y `wm-central` desplegados en Railway.

Comprobación de la parte cloud

Antes de lanzar los módulos, desde un PC del laboratorio:

1. Se abre el panel en el navegador. Deben aparecer las estaciones en gris («Desconectada»).
2. Se comprueba que Kafka es accesible desde fuera de Railway. El comando debe listar los cinco topics `wm.*`:

```
docker run --rm apache/kafka:4.3.1 /opt/kafka/bin/kafka-topics.sh \
    --bootstrap-server altaria.proxy.rlwy.net:55723 --list
```

3. Se comprueba que el servidor de sockets de la central es accesible:

```
timeout 5 bash -c '</dev/tcp/iriguchi.proxy.rlwy.net/26572' && echo
↪  "Central accesible"
```

7.4. Laboratorio

En cada PC del laboratorio, una sola vez:

```
git clone https://github.com/pedrocrosua/water_management_network.git
cd water_management_network
```

Cada aplicación se lanza en su propia terminal y en primer plano, con los scripts de la carpeta `scripts/`. Antes de lanzar la aplicación, el script construye su imagen de Docker. La primera vez

compila el proyecto con Maven y tarda unos minutos; después, mientras no cambie el código, Docker reutiliza lo ya construido y la aplicación arranca al momento.

Por eso, **antes de la corrección**, se lanza una vez en cada PC cada tipo de aplicación (un Monitor y un Engine en el de las estaciones, y un `WM_FO` en el de los operarios) y se cierra con `Ctrl+C`. Así, durante la corrección las imágenes ya están construidas y nada se compila.

PC de las estaciones

Cada estación necesita dos terminales: primero se lanza su Monitor y después su Engine.

```
./scripts/lab-ws.sh monitor WS-01 "Parque de Canalejas"   # terminal 1
./scripts/lab-ws.sh engine WS-01                          # terminal 2
./scripts/lab-ws.sh monitor WS-02 "Jardines de Abril"     # terminal 3
./scripts/lab-ws.sh engine WS-02                          # terminal 4
./scripts/lab-ws.sh monitor WS-03 "Parque El Palmeral"    # terminal 5
./scripts/lab-ws.sh engine WS-03                          # terminal 6
```

Cada estación es un proyecto de Docker Compose independiente (`wm-ws-01`, `wm-ws-02`…), con una red privada propia en la que el Engine encuentra a su Monitor. Los contenedores se llaman `WM_WS_M_<id>` y `WM_WS_E_<id>`.

PC de los operarios

Una terminal por operario:

```
./scripts/lab-fo.sh FO-001   # terminal 1
./scripts/lab-fo.sh FO-002   # terminal 2
```

El contenedor se llama `WM_FO_<id>` y tiene montada la carpeta `requests/`, con el fichero de ejemplo para el modo fichero.



Figura 3: Contenedores en ejecución en el PC de las estaciones (`docker ps`).

Pruebas durante la corrección

Prueba	Cómo se hace
Ver el estado de las estaciones	Panel web de la central, u opción 1 de un `WM_FO`
Pedir un riego	Opción 2 de un `WM_FO`: estación y segundos
Peticiones desde un fichero	Opción 3 de un `WM_FO`, con `requests/example.csv`
Riego ordenado por la central	Botones `Regar` o `Regar todas` del panel; los segundos se indican en la cabecera
Bloquear y activar estaciones	Botones `Bloquear`, `Activar`, `Bloquear todas` y `Activar todas` del panel
Avería y reparación	Opciones 1 y 2 del `WM_WS_E`
Parar un riego desde la estación	Opción 3 del `WM_WS_E`
Riego manual	Opción 4 del `WM_WS_E`
Caída de un módulo	`Ctrl+C` o cerrar su terminal. Para recuperarlo, se vuelve a lanzar con el mismo comando
Estación nueva	Se lanzan un Monitor y un Engine con otro identificador, por ejemplo `WS-04`: la central la da de alta al conectarse

Para parar todo, basta con cerrar las terminales: los scripts borran sus contenedores al terminar.

Alternativa sin Railway

Solo sirve para seguir probando el sistema si Railway fallara. No sustituye al escenario de la corrección, que exige llegar con la parte cloud desplegada y funcionando en Railway.

La central y Kafka se pueden ejecutar en un PC del aula. En ese PC:

```
PUBLIC_HOST=<ip del PC> docker compose up -d --build kafka wm-central
```

En el fichero `lab.env` de los demás PCs se ponen las direcciones de ese equipo:

```
CENTRAL_HOST=<ip del PC>
CENTRAL_PORT=5000
KAFKA_BOOTSTRAP=<ip del PC>:29092
```

El panel queda en `http://<ip del PC>:8080`.

8. Resultados

8.1. Pruebas automáticas

El proyecto tiene 240 pruebas automáticas con JUnit 6, que se ejecutan dentro de Docker con `docker compose run --rm maven mvn -B test`. Todas pasan:

Módulo	Pruebas
`wm-common`	91
`wm-central`	63
`wm-ws-engine`	41
`wm-ws-monitor`	23
`wm-field-operator`	22
Total	**240**

Cubren el protocolo de sockets (tramas, LRC, ACK/NACK y reintentos), las conexiones por socket entre los módulos, la lógica de la central (validación y autorización de riegos, averías, bloqueos y caídas), el Engine (órdenes, reloj del riego y copia del riego en curso) y el operario (peticiones, notificaciones y modo fichero).

8.2. Pruebas del sistema desplegado

El escenario completo, con la central y Kafka en Railway y las estaciones y los operarios en los PCs del laboratorio, se ha probado en el aula. Las caídas de cada módulo se han probado con Docker: en ningún caso se pierde un riego y solo hay que volver a lanzar el módulo caído.

Las capturas siguientes corresponden a una ejecución del escenario completo, con la parte cloud en Railway, tres estaciones y dos operarios.

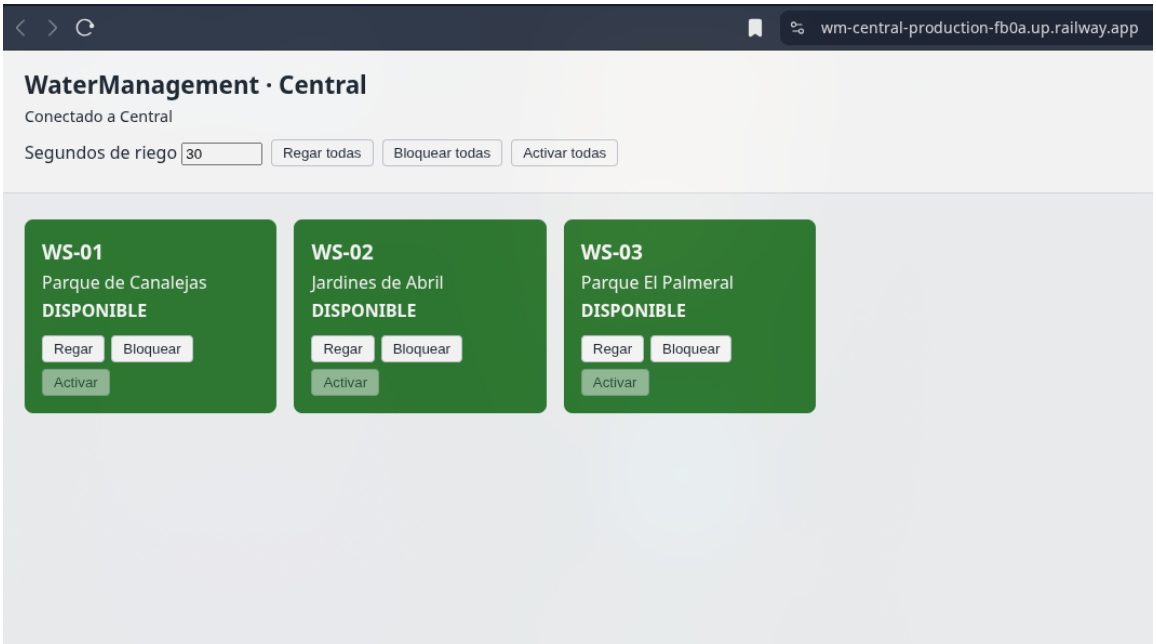


Figura 4: Panel de `WM_Central` con las estaciones conectadas.



Figura 5: Engine de `WS-03` (izquierda) y Monitor de `WS-01` (derecha), cada uno en su terminal.

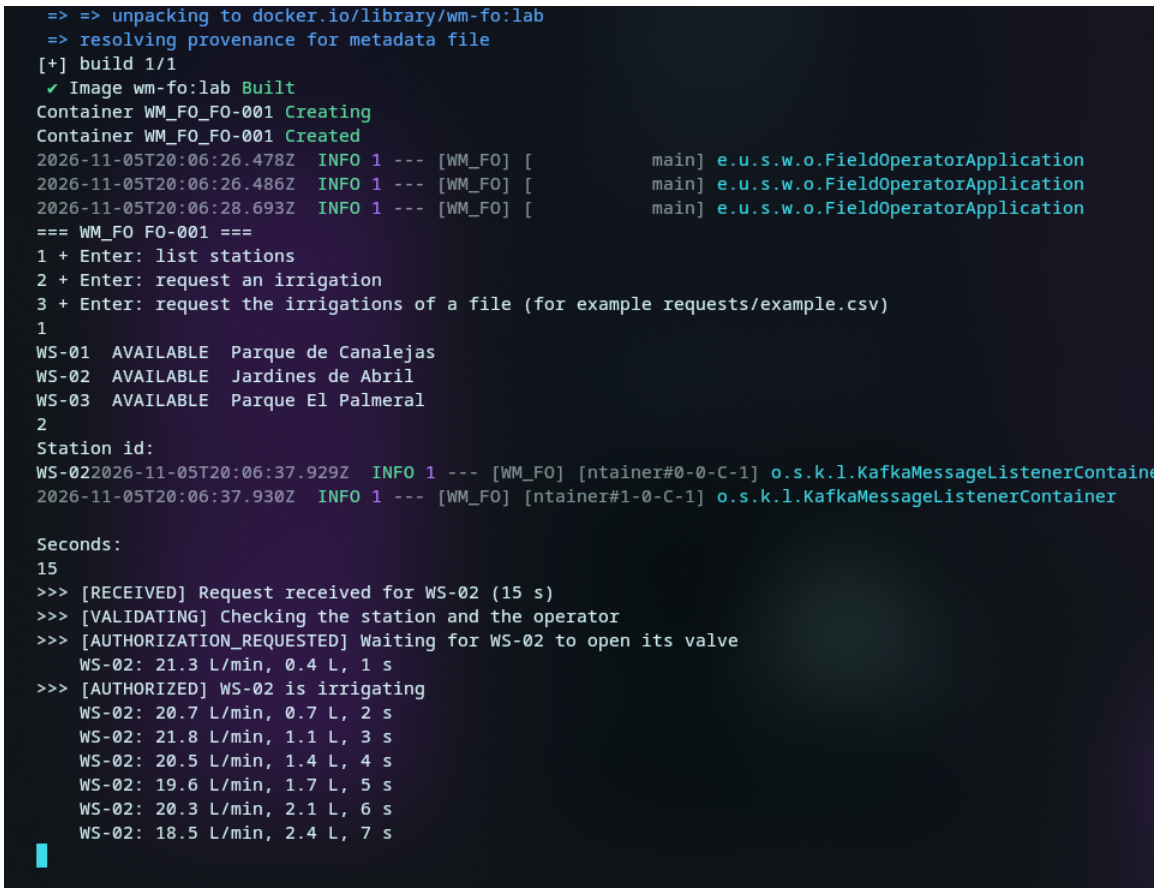


Figura 6: WM_FO pidiendo un riego: pasos de la petición y progreso segundo a segundo.

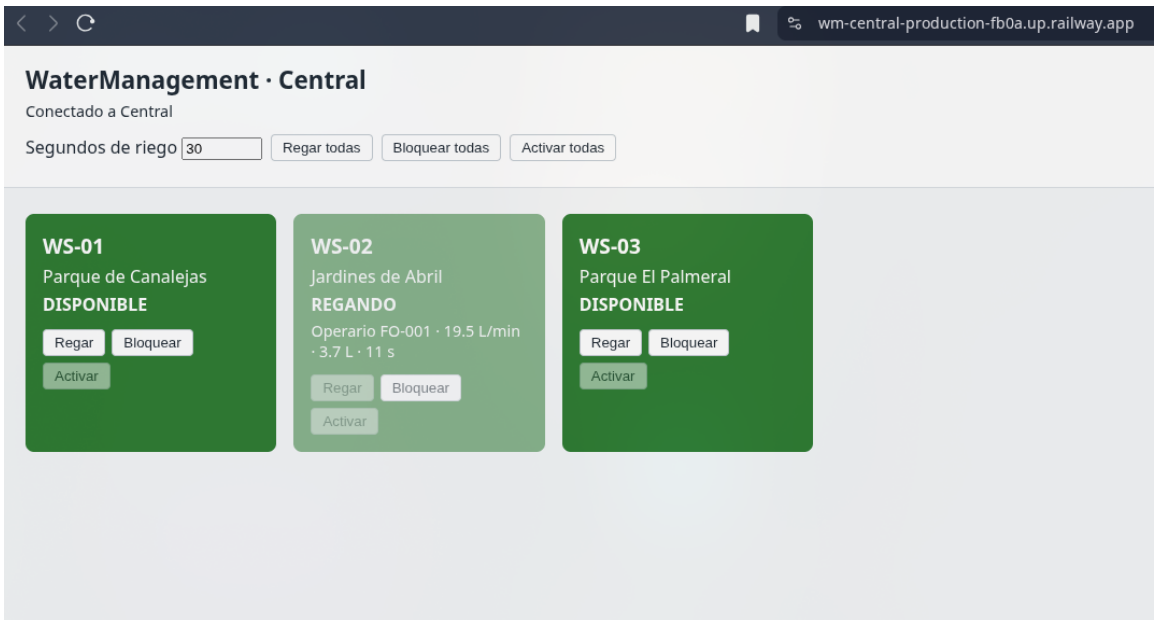


Figura 7: Panel con una estación regando: caudal, litros acumulados y operario.

```
=== WM_FO FO-001 ===
1 + Enter: list stations
2 + Enter: request an irrigation
3 + Enter: request the irrigations of a file (for example requests/example.csv)
2
Station id:
WS-02
Seconds:
15
>>> [RECEIVED] Request received for WS-02 (15 s)
>>> [VALIDATING] Checking the station and the operator
>>> [AUTHORIZATION_REQUESTED] Waiting for WS-02 to open its valve
>>> [AUTHORIZED] WS-02 is irrigating
    WS-02: 21.4 L/min, 0.4 L, 1 s
    WS-02: 22.0 L/min, 0.7 L, 2 s
    WS-02: 20.4 L/min, 1.1 L, 3 s
    WS-02: 20.7 L/min, 1.4 L, 4 s
    WS-02: 20.8 L/min, 1.8 L, 5 s
    WS-02: 18.3 L/min, 2.1 L, 6 s
    WS-02: 20.6 L/min, 2.4 L, 7 s
    WS-02: 21.6 L/min, 2.8 L, 8 s
    WS-02: 18.0 L/min, 3.1 L, 9 s
    WS-02: 20.0 L/min, 3.4 L, 10 s
    WS-02: 19.5 L/min, 3.7 L, 11 s
    WS-02: 21.7 L/min, 4.1 L, 12 s
    WS-02: 19.9 L/min, 4.4 L, 13 s
    WS-02: 20.1 L/min, 4.8 L, 14 s
    WS-02: 20.8 L/min, 5.1 L, 15 s
>>> [IRRIGATION_ENDED] WS-02 finished (TIME_LIMIT): 5.1 L in 15 s
```

Figura 8: Resumen final del riego en `WM_FO`: litros, duración y motivo de fin.

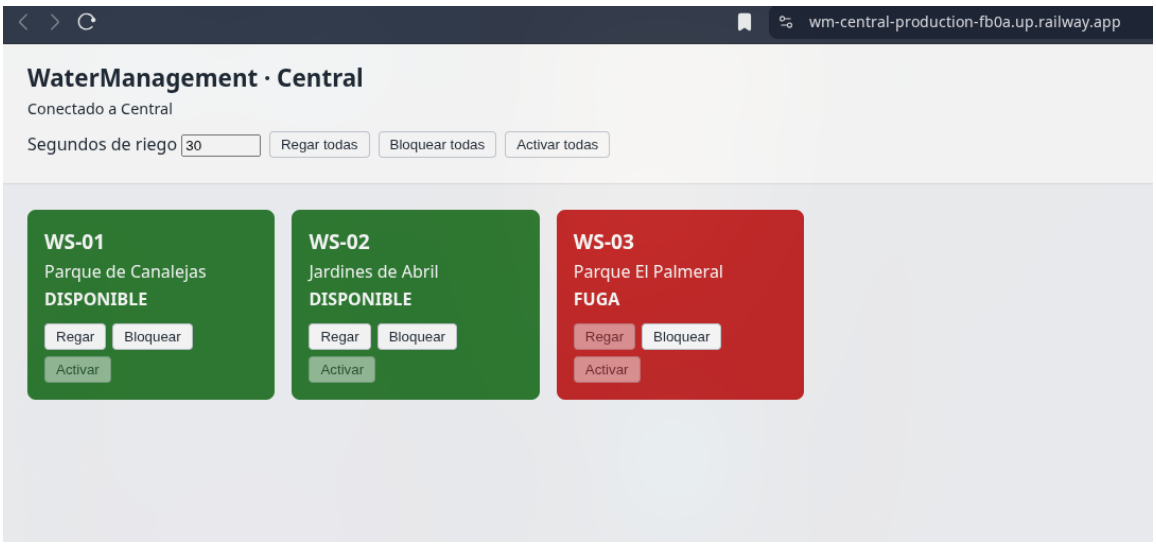


Figura 9: Avería simulada desde el Engine: la estación pasa a rojo y se corta el riego.

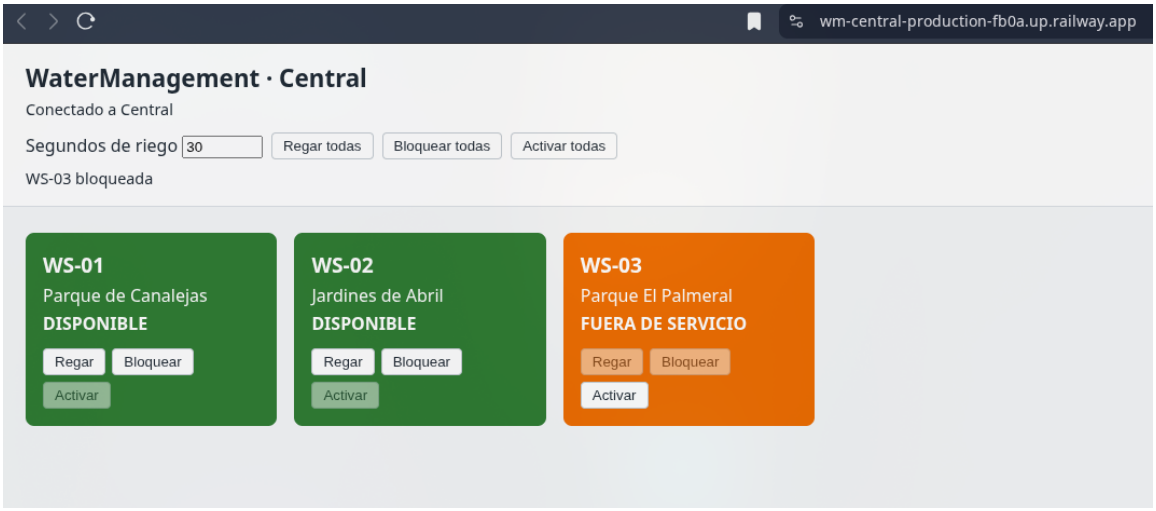


Figura 10: Estación bloqueada desde el panel (naranja, «Fuera de Servicio»).

```
=== WM_FO FO-001 ===
1 + Enter: list stations
2 + Enter: request an irrigation
3 + Enter: request the irrigations of a file (for example requests/example.csv)
3
File:
requests/example.csv
Line 3: asking WS-01 for 10 s
>>> [RECEIVED] Request received for WS-01 (10 s)
>>> [VALIDATING] Checking the station and the operator
>>> [AUTHORIZATION_REQUESTED] Waiting for WS-01 to open its valve
>>> [AUTHORIZED] WS-01 is irrigating
    WS-01: 20.6 L/min, 0.3 L, 1 s
    WS-01: 20.8 L/min, 0.7 L, 2 s
    WS-01: 19.7 L/min, 1.0 L, 3 s
    WS-01: 20.5 L/min, 1.4 L, 4 s
    WS-01: 20.6 L/min, 1.7 L, 5 s
    WS-01: 20.2 L/min, 2.0 L, 6 s
    WS-01: 18.1 L/min, 2.3 L, 7 s
    WS-01: 18.3 L/min, 2.6 L, 8 s
    WS-01: 19.0 L/min, 3.0 L, 9 s
    WS-01: 20.0 L/min, 3.3 L, 10 s
>>> [IRRIGATION_ENDED] WS-01 finished (TIME_LIMIT): 3.3 L in 10 s
Line 4: asking WS-02 for 8 s
>>> [RECEIVED] Request received for WS-02 (8 s)
>>> [VALIDATING] Checking the station and the operator
>>> [AUTHORIZATION_REQUESTED] Waiting for WS-02 to open its valve
>>> [AUTHORIZED] WS-02 is irrigating
    WS-02: 21.6 L/min, 0.4 L, 1 s
    WS-02: 19.9 L/min, 0.7 L, 2 s
    WS-02: 19.8 L/min, 1.0 L, 3 s
    WS-02: 18.6 L/min, 1.3 L, 4 s
    WS-02: 20.0 L/min, 1.7 L, 5 s
    WS-02: 20.9 L/min, 2.0 L, 6 s
    WS-02: 21.2 L/min, 2.4 L, 7 s
    WS-02: 19.8 L/min, 2.7 L, 8 s
>>> [IRRIGATION_ENDED] WS-02 finished (TIME_LIMIT): 2.7 L in 8 s
Line 5: asking WS-01 for 5 s
>>> [RECEIVED] Request received for WS-01 (5 s)
>>> [VALIDATING] Checking the station and the operator
>>> [AUTHORIZATION_REQUESTED] Waiting for WS-01 to open its valve
>>> [AUTHORIZED] WS-01 is irrigating
    WS-01: 19.1 L/min, 0.3 L, 1 s
    WS-01: 18.0 L/min, 0.6 L, 2 s
    WS-01: 19.0 L/min, 0.9 L, 3 s
    WS-01: 18.4 L/min, 1.2 L, 4 s
    WS-01: 19.3 L/min, 1.6 L, 5 s
>>> [IRRIGATION_ENDED] WS-01 finished (TIME_LIMIT): 1.6 L in 5 s
File finished. Requests sent: 3
```

Figura 11: Peticiones de riego leídas de un fichero en `WM_FO`.

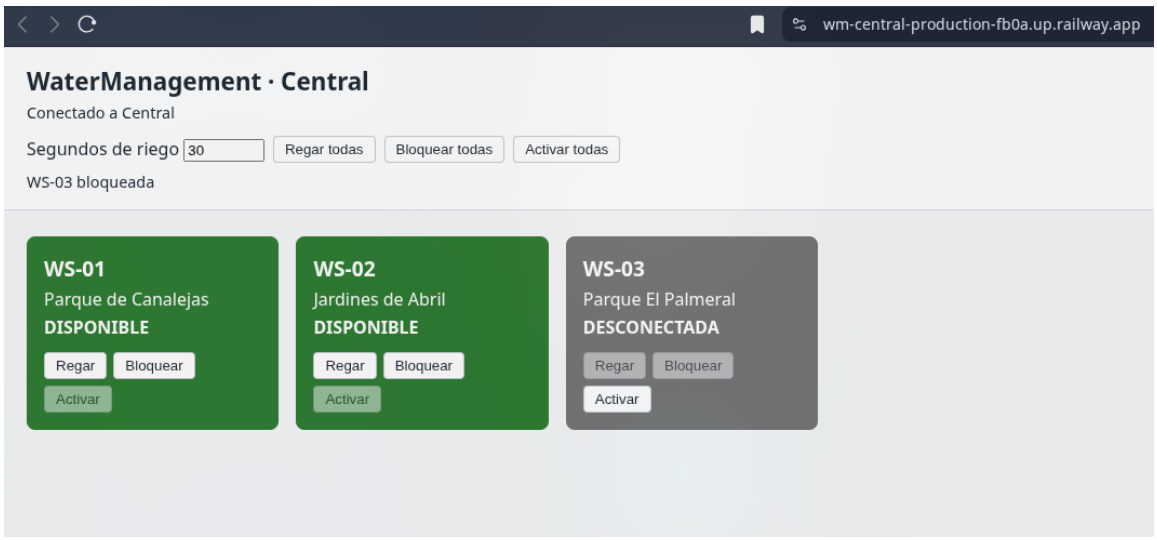


Figura 12: Caída de un Monitor: la estación aparece desconectada hasta que se vuelve a lanzar.

9. Aspectos técnicos destacables

- **Protocolo de sockets con tramas bien formadas** <STX><DATA><ETX><LRC>, con confirmación ACK/NACK, reintentos y detección de caídas por tiempo de espera.
- **Resiliencia.** Ante la caída de cualquier módulo, el sistema sigue funcionando en lo posible, muestra el error de forma controlada y se recupera al volver a lanzar solo el módulo caído. Ningún riego se pierde: el Engine guarda una copia del riego en curso y la central guarda los riegos en marcha.
- **Escalabilidad.** Las estaciones nuevas se dan de alta solas al conectarse, cada estación y cada operario se ejecuta en su propio proyecto de Docker Compose (se pueden lanzar varios en el mismo PC) y los grupos de consumidores de Kafka permiten añadir instancias sin cambiar el código.
- **Un único `Dockerfile`** para las cuatro aplicaciones y ninguna dependencia en los PCs del laboratorio, aparte de Docker y git.
- **Kafka con dos *listeners* para los clientes**, uno interno y otro externo, para que funcione a la vez dentro de Railway y desde el laboratorio.

10. Conclusiones

La práctica ha permitido comparar en un mismo sistema dos formas de comunicación. Los sockets con protocolo propio obligan a definir cada trama y a gestionar las confirmaciones, los reintentos y los

tiempos de espera, pero permiten detectar al momento la caída del otro extremo. Kafka, en cambio, desacopla a quien publica de quien consume y facilita añadir instancias, aunque obliga a pensar bien los grupos de consumidores y desde qué mensaje empieza a leer cada aplicación.

Las mayores dificultades han sido el despliegue de Kafka en Railway, para que fuera accesible a la vez desde la red privada y desde el laboratorio, y mantener un estado coherente cuando cualquier módulo se cae en mitad de un riego.